

ROS2 Day2-hw3 과제 보고서

로봇 21기 예비단원 최윤서

1. 어느 부분에서 사용했는가?

- Publish랑 subscribe까진 Day1에서 구현했는데 QT GUI랑 어떻게 연결해야하는지 공부하려고했습니다.
- <https://blog.naver.com/kswdavid/223051746409>
- 위의 링크에서 QT와 ROS2 Node를 연동하는 예제 코드를 찾을 수 있었습니다.
- 코드를 이해하고 제대로 이해한 것이 맞는지 확인하기 위해 AI에게 코드 구조와 작동 과정을 물어봤습니다.

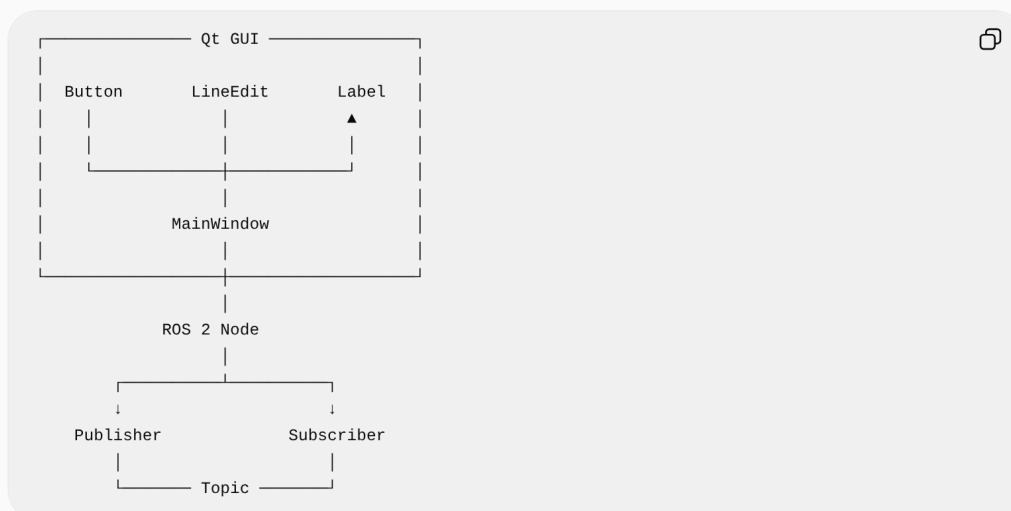
<https://blog.naver.com/kswdavid/223051746409>

여기서 ROS2랑 QT GUI 연동하는 방법 코드 구조 이해할 수 있게 도와줘

응. 다만 해당 네이버 블로그 본문 자체는 웹에서 직접 열리지 않아서, 블로그의 코드를 그대로 분석해서 설명해 드릴게요. 대신 ROS 2 + Qt 연동에서 사용하는 핵심 구조와 네가 구현하려는 구조를 기준으로 코드가 왜 그렇게 구성되는지 이해할 수 있게 설명할게요. ROS 2와 Qt를 결합할 때는 Qt 이벤트 루프와 ROS 2의 `spin()` 처리를 함께 고려해야 한다는 점이 핵심입니다. Robotics Stack E... +1 Upgrade

1. 먼저 전체 구조부터 이해하면 돼

ROS 2와 Qt를 연결할 때 크게 두 영역으로 나눠서 생각해.



즉 GUI 자체가 ROS 2 통신을 하는 것으로 생각하기보다, GUI와 ROS 2 Node를 연결한다고 생각하면 이해하기 쉽다.

2. 왜 MainWindow 와 Node 를 분리하느냐

역할이 다르기 때문이야.

Qt 쪽

```
MainWindow
├─ 버튼
├─ 입력창
├─ Label
└─ 화면 업데이트
```



ROS 2 쪽

```
Node
├─ Publisher
├─ Subscriber
├─ Topic
└─ Callback
```



ROS 2 공식 생태계에서도 Qt 기반 GUI를 ROS 기능과 결합하는 구조가 사용되며, GUI와 ROS 통신 계층을 분리하는 방식은 실제 ROS 2 GUI 프로젝트에서도 사용된다. [GitHub +1](#)

✦ Upgrade

3. Publisher를 GUI 버튼과 연결

예를 들어 GUI에

```
[ 입력창 ] [Send]
```



가 있다고 하자.

사용자가 Hello 를 입력하고 Send를 누르면:

```
Send 버튼
┆
┆ clicked Signal
┆
┆ MainWindow
┆
┆ 입력창의 문자열 가져오기
┆
┆ ROS 2 Publisher
┆
┆ Topic
```



이런 식으로 연결한다.

여기서 중요한 건 버튼이 직접 Publisher가 되는 게 아니↓
↓

버튼은 Qt 객체고, Publisher는 ROS 2 객체야.

✦ Upgrade

그래서 MainWindow 가 중간에서 둘을 연결한다.

4. Subscriber는 반대 방향

Subscriber는 다음과 같아.

```
Topic
┆
┆ ROS 2 Subscriber
┆
┆ Callback
┆
┆ MainWindow에 전달
┆
┆ GUI 위젯 변경
```



예를 들어 다른 노드에서:

```
/chatter
┆
┆ "Hello"
```



를 Publish했다고 하자.



Subscriber callback에서 "Hello" 를 받는다.

Upgrade

그런데 callback 안에서 바로 GUI의 QLabel 을 건드리는 구조보다는,

```
Subscriber callback
|
Qt Signal
|
MainWindow
|
QLabel
```

처럼 ROS 2 영역에서 Qt 영역으로 데이터를 전달하는 구조가 이해하기 좋다.

5. 여기서 Qt의 Signal/Slot이 등장한다

Upgrade

이 부분이 처음 보면 가장 헷갈려.

ROS 2의 Subscriber가:

```
"Hello"
```

를 받았다고 하자.

그걸 Qt GUI에 전달하려면 **신호를 하나 보내는** 것으로 생각하면 된다.

```
Subscriber
|
"Hello"
|
Signal 발생
|
MainWindow가 받음
|
Label에 "Hello" 표시
```

Qt의 Signal/Slot은 객체 사이의 이벤트와 데이터 전달을 연결하는 메커니즘이다.

따라서 개념적으로는:

```
ROS 2 callback = "메시지가 도착했다"
```

```
Qt Signal = "GUI 쪽에 이 메시지가 도착했다고 알려준다"
```

```
Qt Slot/연결된 함수 = "그러면 화면을 업데이트한다"
```

라고 이해하면 된다.

6. 그런데 ROS 2의 spin() 은 어디서 처리하느냐

Upgrade

여기가 ROS 2 + Qt 연동에서 가장 중요한 부분이다.

Qt 프로그램에는:

```
app.exec()
```

이라는 Qt 이벤트 루프가 필요하다.

반면 ROS 2 Subscriber callback을 처리하려면:

```
rcicpp::spin()
```

같은 ROS 2 실행 처리가 필요하다.

문제는 둘 다 프로그램의 이벤트를 처리를 계속 담당한다는 것이다. ROS 2와 Qt를 함께 사용할 때 이 두 이벤트 루프를 어떻게 통합할지가 핵심적인 문제로 알려져 있다. [Stack Overflow](#)

그래서 구현 방식 중 하나가:

```
Qt 이벤트 루프
+
ROS 2 Executor
```

를 같이 돌리는 것이다.

Upgrade

7. spin_some() 방식으로 생각하면 이해가 쉽다

예를 들어 Qt의 QTimer가 있다고 하자.

```
QTimer
├─ 주기적으로 실행
├─
├─ ROS 2 Executor
├─
├─ spin_some()
├─
└─ 대기 중인 ROS 2 callback 처리
```



그러면 전체 구조가:

Upgrade

```
Qt Event Loop
├─
├─ 버튼 이벤트
├─
├─ 화면 업데이트
├─
├─ QTimer
├─   └─ spin_some()
├─       └─
├─           └─ ROS 2 callback
├─               └─
├─                   └─ Qt Signal
├─                       └─
├─                           └─ GUI 업데이트
├─
└─ 기타 Qt 이벤트
```



가 된다.

이 방식은 ROS 2의 spin()으로 Qt 이벤트 루프를 막아버리지 않고, Qt가 계속 동작하면서 일정 주기마다 ROS 2 callback을 처리한다는 개념이다.

Upgrade

8. 그래서 전체 코드를 역할별로 보면

프로젝트 파일이 다음처럼 나뉘어 있다고 생각하면 된다.

```
main.cpp
├─ 프로그램 시작
├─   └─ QApplication
├─   └─ rclcpp 초기화
├─   └─ MainWindow 실행

mainwindow.cpp
├─ GUI 생성
├─ Publisher 연결
├─ Subscriber 연결
├─ ROS Executor 처리

talker_node.cpp
├─ Publisher
├─   └─ Topic으로 메시지 전송
```



```
listener_node.cpp
├─ Subscriber
├─   └─ Callback
├─       └─ Qt로 데이터 전달
```

Upgrade

이렇게 파일별 역할을 분리해서 보면 코드가 훨씬 단순해진다.

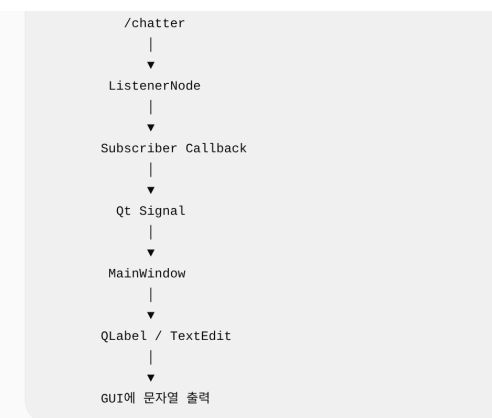
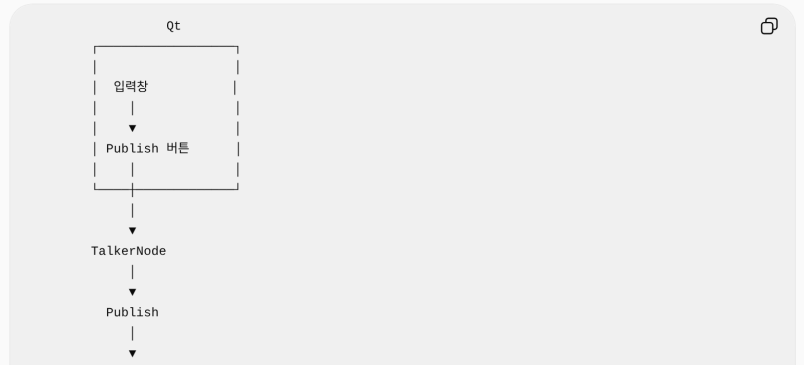
9. 네가 지금 구현하려는 기능을 이 구조에 넣으면

Upgrade

목표가:

Publish 버튼으로 문자열을 보내고, Subscribe한 문자열을 GUI에 출력

이러면 구조는 정확히 이것이다.



그리고 이 전체에서 ROS 2 callback이 작동하도록:



그리고 이 전체에서 ROS 2 callback이 작동하도록:

Upgrade

```
QTimer
↓
Executor
↓
spin_some()
```

가 돌아가야 한다.

핵심만 정리하면

ROS 2 ↔ Qt GUI 연동의 핵심은 3개야.

1. Qt GUI와 ROS 2 Node의 역할을 분리한다.
2. ROS 2 Callback에서 받은 데이터를 Qt Signal/Slot을 통해 GUI로 전달한다.
3. Qt 이벤트 루프가 돌아가는 동안 ROS 2 Executor도 주기적으로 callback을 처리하게 한다.

이 세 가지를 이해하면, 블로그의 코드가 길어 보여도 "GUI → ROS Publisher"와 "ROS Subscriber → GUI"를 연결한 것으로 구조를 해석할 수 있어. [GitHub +1](#)



- 느낀점)

- 처음에는 ROS 2의 Publisher와 Subscriber 연결을 어떻게 시킬 수 있는건지 잘 몰랐고 예제 코드를 봐도 정확하게 이해하기는 힘들었습니다.
- AI의 답변을 보고 이해할 때와 처음 예제 코드를 이해할 때를 비교해보면 제가 코드에 있는 함수 하나하나를 이해할 수 있지만 전체적으로 구조를 이해하는 능력이 아직 부족하다는 생각이 들었습니다.